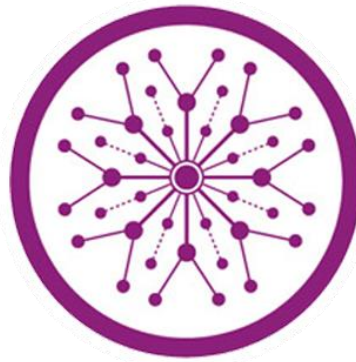


CareerPath ATS Studio

Programming for Artificial Intelligence

Spring 2026

A project submitted in partial fulfilment of the degree of
BS in Artificial Intelligence



Department of Software Engineering

Faculty of Computer Science & Information Technology

The Superior University, Lahore

Spring 2026

TABLE OF CONTENTS

Abstract	i
Chapter 1: Introduction	1
1.1 Introduction	1
1.2 Problem Statement	2
1.3 Proposed Solution	2
1.4 Purpose of the Project	3
1.5 Main Objectives	3
1.6 Scope of the Project	4
Chapter 2: Literature Review and Existing System	5
2.1 Background Study	5
2.2 Existing Manual Resume Review System	6
2.3 Limitations of Existing Systems	6
2.4 Need for the Proposed System	7
Chapter 3: System Analysis	8
3.1 Overview of Proposed System	8
3.2 Project Modules	9
3.3 Functional Requirements	10
3.4 Non-Functional Requirements	11
3.5 Feasibility Study	12
Chapter 4: Requirement Specification	13
4.1 Hardware Requirements	13
4.2 Software Requirements	14
4.3 Project File Structure	15
4.4 Dependencies Used	16
Chapter 5: System Design	17
5.1 System Architecture	17
5.2 Data Flow Diagram	19
5.3 Database Design	21
5.4 User Interface Design	22
Chapter 6: Implementation	24
6.1 Development Environment	24
6.2 Backend Implementation	25
6.3 Frontend Implementation	27
6.4 AI Prompt Design	28
6.5 Report Generation	29
Chapter 7: Testing and Results	30
7.1 Testing Strategy	30
7.2 Test Cases	31
7.3 Output Screens	32
7.4 Result Discussion	35

Chapter 8: Conclusion and Future Work	36
8.1 Conclusion	36
8.2 Future Enhancements	37

LIST OF FIGURES

Figure 1.1: Home Page	4
Figure 2.1: Existing vs Proposed System Diagram	7
Figure 3.1: System Workflow	12
Figure 4.1: Project Folder Structure	15
Figure 5.1: System Architecture Diagram	18
Figure 5.2: Data Flow Diagram	20
Figure 5.3: Dashboard UI	22
Figure 5.4: Upload Resume Page	23
Figure 7.1: ATS Score Result	32
Figure 7.2: Improvements Result	33
Figure 7.3: Skill Gap Result	34
Figure 7.4: Roadmap Result	35

LIST OF TABLES

Table 3.1: Project Modules	9
Table 4.1: Hardware Requirements	13
Table 4.2: Software Requirements	14
Table 4.3: Project File Structure	15
Table 5.1: System Architecture Layers	18
Table 5.2: In-Memory Data Storage Fields	21
Table 6.1: Backend Functions and Routes	26
Table 6.2: Frontend Files and Implementation Details	27
Table 6.3: AI Prompt Stages	28
Table 7.1: Test Cases	31

Abstract

The AI Resume Analyzer is a web-based application developed to assist students, job seekers, and internship applicants in improving their resumes according to modern hiring standards. Many applicants face rejection because their resumes are poorly formatted, lack relevant keywords, or do not clearly present projects and skills. This system solves the problem by accepting a resume file, extracting its text, analyzing it through structured AI prompts, and returning a detailed ATS score with improvement suggestions.

The project is implemented using Flask for the backend, HTML, CSS, and JavaScript for the frontend, and supporting Python libraries for document parsing and report generation. The system supports PDF and DOCX resume uploads, performs step-by-step analysis, identifies missing skills, produces a practical learning roadmap, and allows users to download improved resume output and roadmap reports. The documentation follows a Final Year Project style and includes placeholders where screenshots of code, interface, testing, and outputs can be attached later.

Chapter 1

Introduction

1.1 Introduction

In the current digital job market, resumes are often screened by Applicant Tracking Systems before they are reviewed by recruiters. A resume that does not contain suitable keywords, proper formatting, relevant projects, or clear achievements may fail during the initial screening stage. Students and fresh graduates especially face difficulty because they may not know how to present their skills according to industry requirements.

The AI Resume Analyzer is designed as a smart web-based system that evaluates resumes and provides career-oriented guidance. It helps users understand the quality of their resume, the missing skills required for internship or entry-level roles, and the steps needed to become job-ready. The system provides a structured flow: upload resume, generate ATS score, show improvements, identify skill gaps, generate roadmap, and download output reports.

1.2 Problem Statement

Many students and job seekers submit resumes without knowing whether their document is suitable for ATS screening or recruiter review. Manual resume review is time-consuming and may not provide consistent feedback. Existing online tools often give generic suggestions and do not provide a complete learning roadmap. Therefore, there is a need for a system that can automatically analyze resume content, highlight weaknesses, suggest improvements, and guide the user toward internship readiness.

1.3 Proposed Solution

The proposed solution is an AI-powered resume analysis web application. The system extracts text from uploaded PDF or DOCX resumes and sends the extracted content to an AI model through strict prompt templates. The response is returned in structured JSON format and displayed on a modern dashboard. The user can move through multiple analysis stages and download improved resume documents and skill roadmaps.

1.4 Purpose of the Project

- To provide automated ATS-style resume evaluation.
- To identify resume weaknesses and missing sections.
- To suggest practical improvements for internship and entry-level job applications.
- To identify skill gaps based on the user resume.
- To generate a learning roadmap for career improvement.
- To provide downloadable PDF and DOCX reports for users.

1.5 Main Objectives

1. Design a user-friendly web interface for resume upload and analysis.
2. Extract text from PDF and DOCX resume files.
3. Integrate an AI model API for resume scoring and guidance.
4. Generate ATS score, breakdown, improvements, skill gap, and roadmap.
5. Provide downloadable improved resume and roadmap files.
6. Maintain a clean modular structure suitable for academic demonstration.

1.6 Scope of the Project

The scope of the project is limited to resume analysis and career-readiness guidance. It does not replace a professional recruiter, but it provides useful feedback that can help users improve their resume before applying for internships or jobs. The project can be used by students, fresh graduates, career centers, and training institutes.

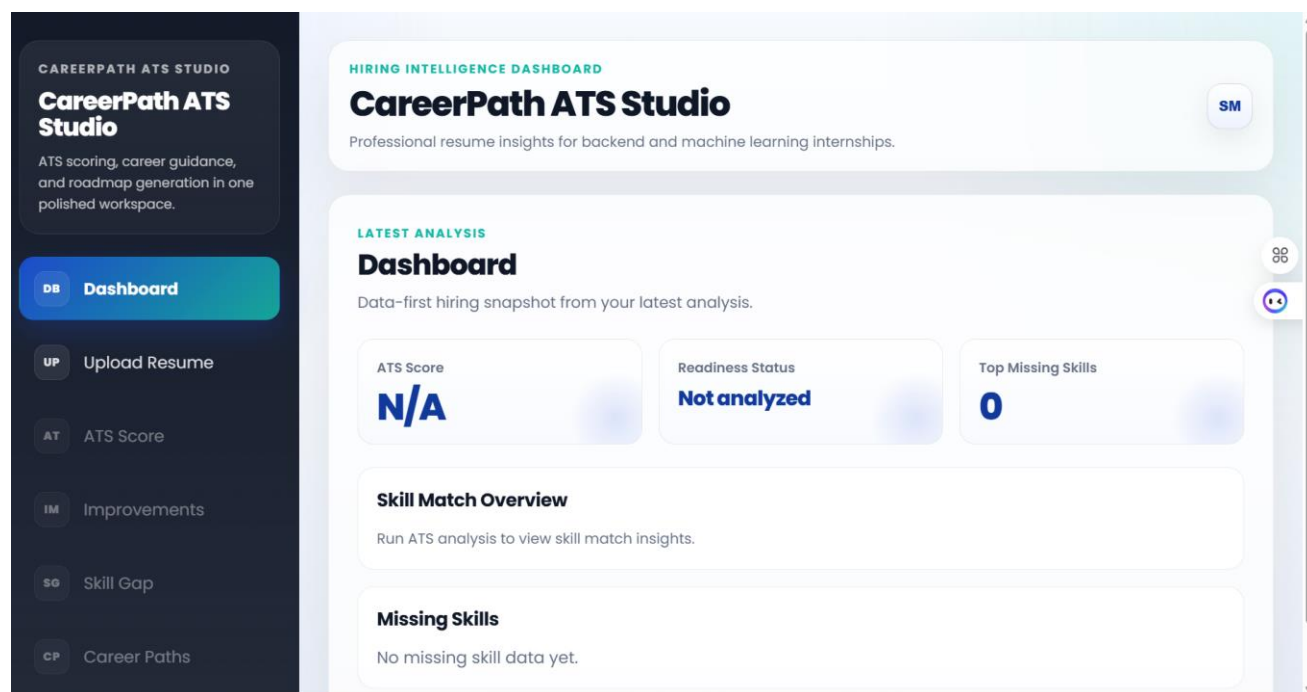


Figure 1.1: Home page

Chapter 2

Literature Review and Existing System

2.1 Background Study

Recruitment has increasingly shifted toward digital platforms where resumes are collected, filtered, and ranked electronically. Applicant Tracking Systems help organizations manage a large number of applications. However, these systems also create challenges for applicants because resumes must be structured clearly and must include job-relevant keywords.

2.2 Existing Manual Resume Review System

In a manual review process, a teacher, mentor, or recruiter reviews a resume and gives feedback. Although this method can be useful, it is slow and depends heavily on the reviewer's availability and expertise. Manual checking may also miss ATS-specific issues such as keyword gaps, formatting problems, and weak project descriptions.

2.3 Limitations of Existing Systems

- Manual review consumes time and effort.
- Feedback may be inconsistent from one reviewer to another.
- Many tools provide only generic comments instead of detailed guidance.
- Some systems do not support downloadable improved resume output.
- Most tools do not generate a structured learning roadmap for missing skills.

2.4 Need for the Proposed System

The proposed system addresses these limitations by providing a complete resume improvement workflow. It combines resume parsing, AI-based analysis, ATS scoring, missing skill detection, roadmap generation, and output file generation in a single web application.

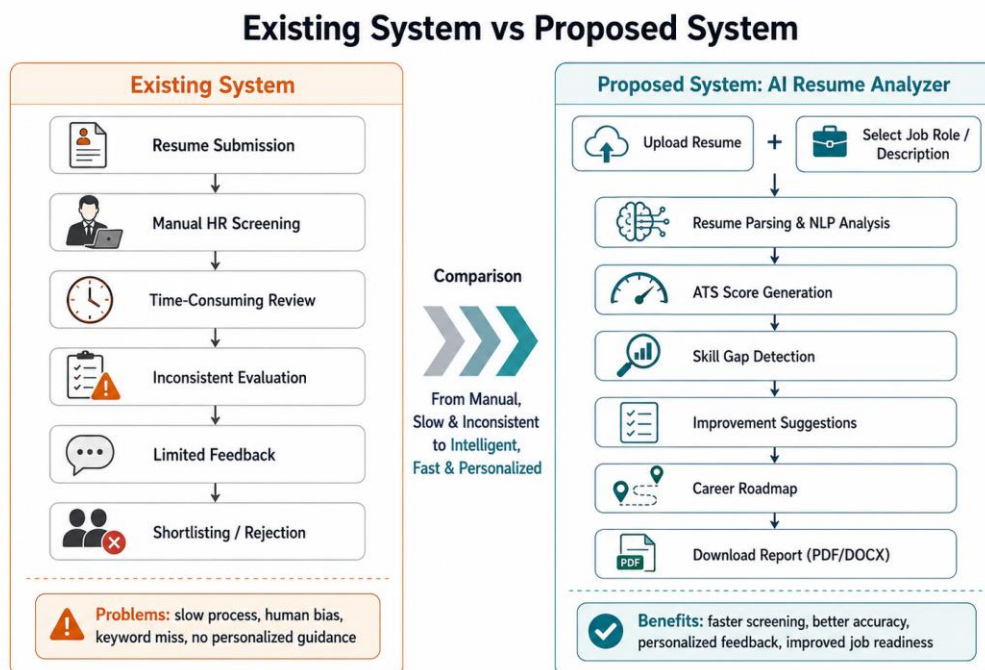


Figure 2.1: Existing System vs Proposed System

Chapter 3

System Analysis

3.1 Overview of Proposed System

The system follows a client-server architecture. The user interacts with the frontend through a browser. The Flask backend receives file uploads, extracts text from the resume, stores analysis data temporarily in memory, communicates with the AI API, and returns structured results to the frontend. The frontend displays the results in cards, buttons, and dashboard sections.

3.2 Project Modules

Module	Description
Resume Upload Module	Accepts PDF and DOCX files from the user.
Text Extraction Module	Extracts resume text using pypdf and python-docx.
ATS Analysis Module	Generates ATS score and detailed breakdown.
Improvement Module	Suggests formatting, keyword, and content improvements.
Skill Gap Module	Identifies missing technical and employability skills.
Roadmap Module	Creates an 8 to 12 week learning roadmap.
Download Module	Generates improved resume in PDF/DOCX and roadmap in PDF.
Template Suggestion Module	Provides resume formatting and ATS optimization suggestions.

3.3 Functional Requirements

- 7. The system shall allow users to upload a resume file in PDF or DOCX format.
- 8. The system shall extract readable text from the uploaded resume.
- 9. The system shall generate an ATS score and score breakdown.
- 10. The system shall generate resume improvement suggestions.
- 11. The system shall generate skill gap analysis.
- 12. The system shall generate a learning roadmap.
- 13. The system shall provide downloadable reports.
- 14. The system shall show error messages for invalid or unsupported files.

3.4 Non-Functional Requirements

- Usability: The interface should be simple and easy to understand.
- Reliability: The system should handle invalid files and API errors gracefully.
- Performance: Resume processing should complete within a reasonable time.
- Maintainability: Code should be organized into clear functions and modules.
- Security: API keys should be stored in environment variables and not exposed in frontend code.

3.5 Feasibility Study

The project is technically feasible because it uses widely available open-source Python libraries and a lightweight Flask framework. It is operationally feasible because users only need a browser and a resume file. It is economically feasible because no specialized hardware is required for demonstration and deployment.

System Workflow – AI Resume Analyzer

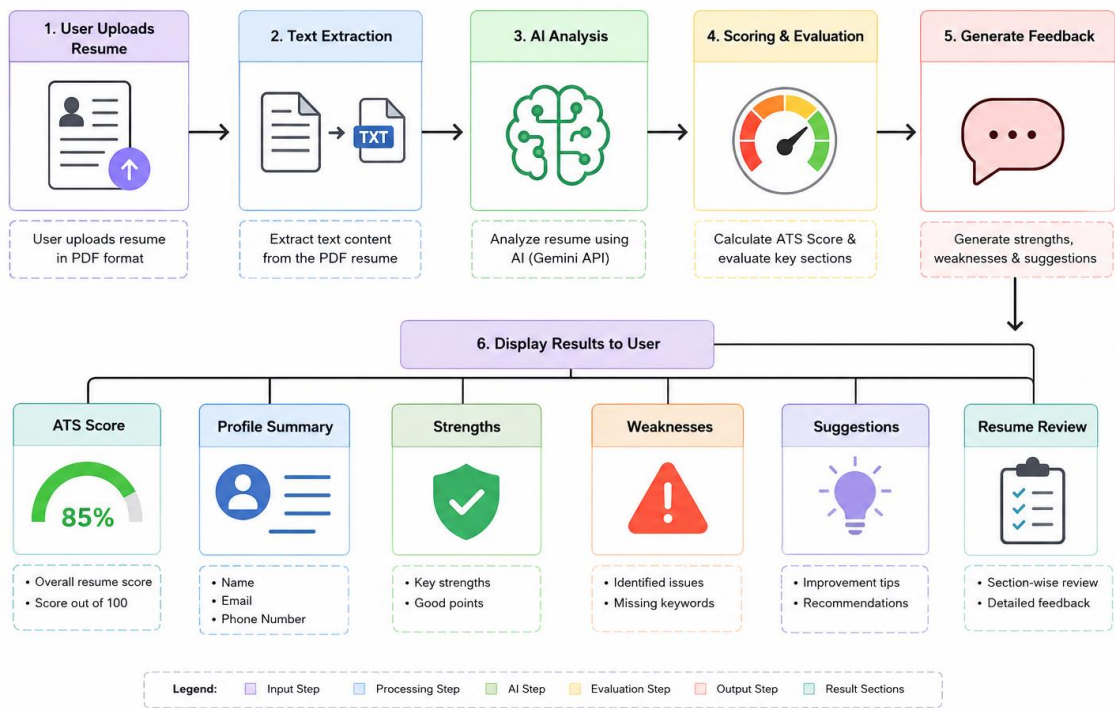


Figure 3.1: System Workflow

Chapter 4

Requirement Specification

4.1 Hardware Requirements

Requirement	Minimum Specification
Processor	Intel Core i3 or equivalent
RAM	4 GB or above
Storage	500 MB free space for project and dependencies
Network	Internet connection for AI API access

4.2 Software Requirements

Software / Tool	Purpose
Python 3.x	Backend programming language
Flask 3.0.3	Web application framework
HTML, CSS, JavaScript	Frontend interface
pypdf 4.3.1	PDF text extraction
python-docx 1.1.2	DOCX text extraction and DOCX generation
reportlab 4.0.9	PDF report generation
requests 2.32.3	HTTP communication with AI API
python-dotenv 1.0.1	Environment variable loading

4.3 Project File Structure

File / Folder	Purpose
app.py	Main Flask backend, routes, prompts, API calls, extraction, and exports.
templates/index.html	Main HTML layout and dashboard structure.
static/app.js	Frontend logic for upload, stage analysis, display, and downloads.
static/styles.css	Styling and responsive interface design.
requirements.txt	List of required Python packages.
.env	Environment configuration such as API keys.
test_resume.txt	Sample resume text used for testing.

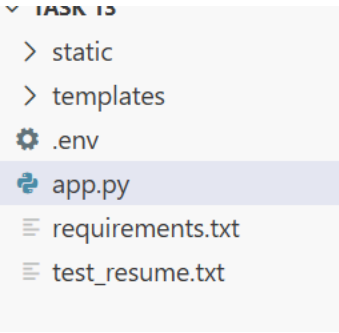


Figure 4.1: Project Folder Structure

4.4 Dependencies Used

The project requirements file includes Flask, requests, pypdf, python-docx, python-dotenv, and reportlab. These packages are enough to run the application, process documents, communicate with the AI API, and generate downloadable reports.

Chapter 5

System Design

5.1 System Architecture

The architecture consists of a frontend browser interface, a Flask backend server, document parsing functions, AI model communication, temporary in-memory result storage, and report generation functions. The frontend sends requests to the backend using fetch API calls. The backend returns JSON responses, which are displayed dynamically in the dashboard.

Layer	Components	Function
Presentation Layer	HTML, CSS, JavaScript	Displays dashboard, upload form, analysis results, and download buttons.
Application Layer	Flask routes and Python functions	Handles requests, stage execution, validation, and business logic.
AI Processing Layer	Prompt templates and AI API request	Generates ATS score, improvements, skill gaps, and roadmap.
Document Layer	pypdf, python-docx, reportlab	Extracts resume text and creates downloadable documents.

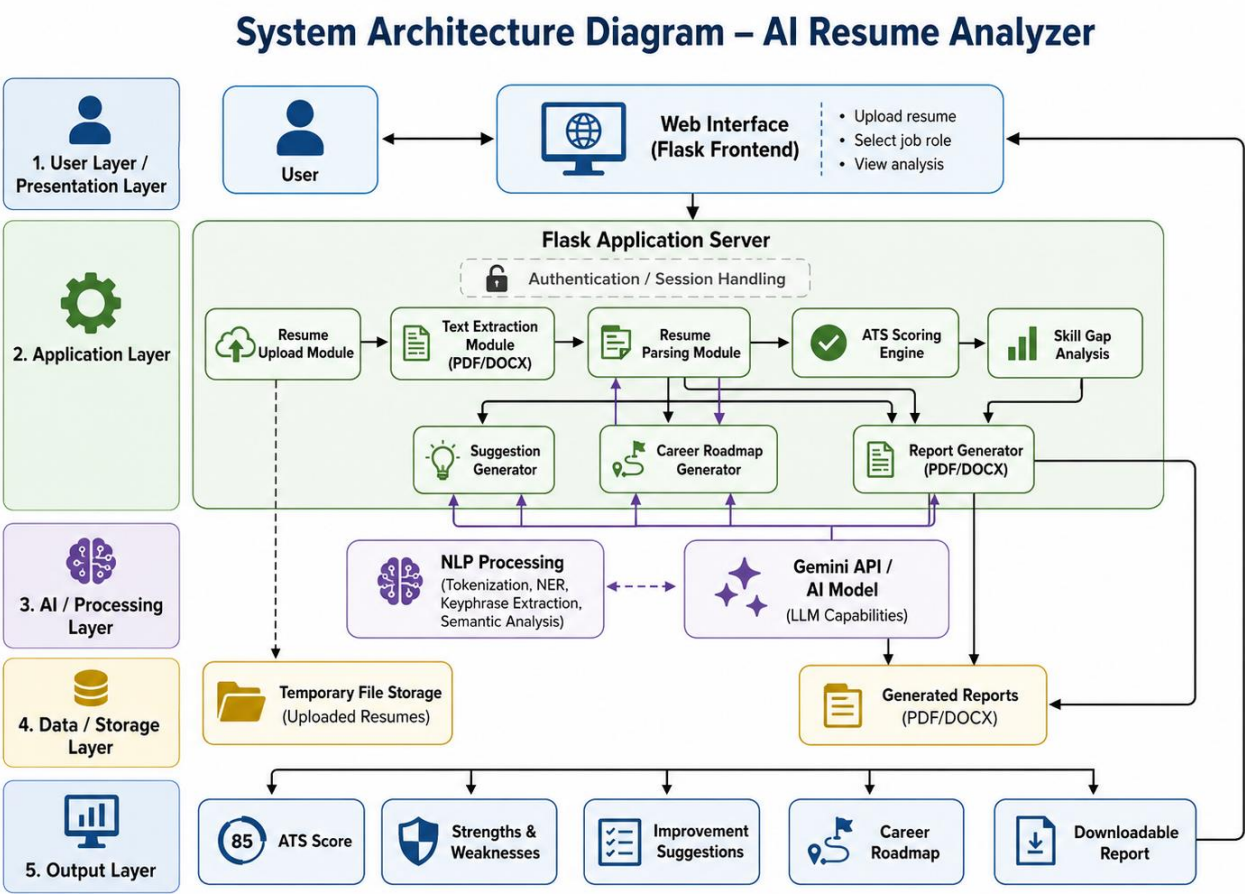


Figure 5.1: System Architecture Diagram

5.2 Data Flow

- 15. User opens the AI Resume Analyzer web interface.
- 16. User uploads a PDF or DOCX resume.
- 17. Backend extracts text from the uploaded resume.
- 18. Backend creates an analysis ID and stores resume text temporarily.
- 19. ATS prompt is sent to the AI model and result is returned.
- 20. User can run improvement, skill gap, and roadmap stages.
- 21. Frontend displays each stage result and updates the dashboard.
- 22. User downloads improved resume or skill roadmap report.

4.4 Data Flow Diagram

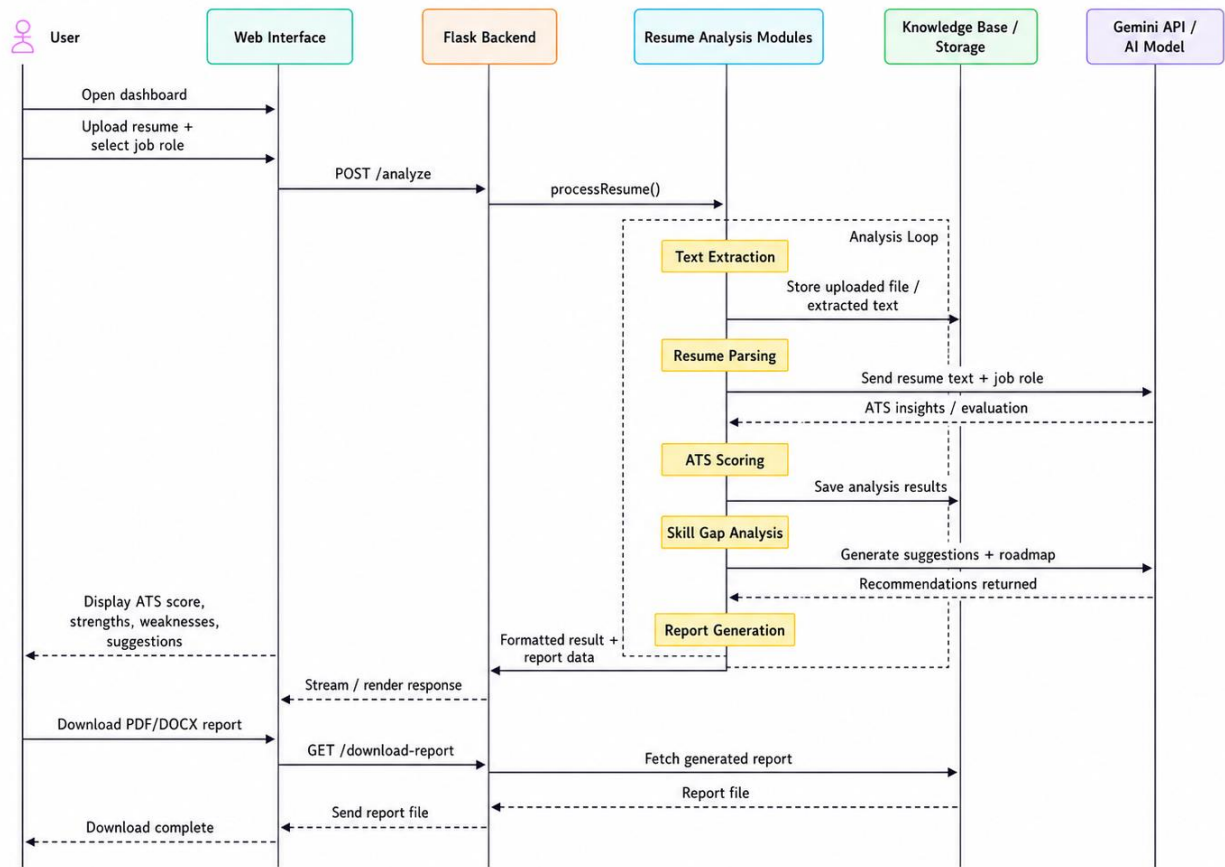


Figure 5.2: Data Flow Diagram

5.3 Database Design

This version of the project does not use a permanent database. It stores uploaded resume text and analysis results temporarily in an in-memory Python dictionary named IN_MEMORY_RESUMES. This is suitable for demonstration purposes, but a future version can use SQLite, MySQL, PostgreSQL, or Firebase for persistent storage.

Field	Description
analysis_id	Unique UUID assigned to each uploaded resume.
resume_text	Extracted text from uploaded resume.
results	Dictionary containing ATS, improvements, skill_gap, and roadmap outputs.

5.4 User Interface Design

The interface contains a sidebar navigation with Dashboard, Upload Resume, ATS Score, Improvements, Skill Gap, and Roadmap sections. It also contains dashboard cards for ATS score, readiness status, and missing skills. The user interface is designed to be clean, responsive, and simple for students and job seekers.

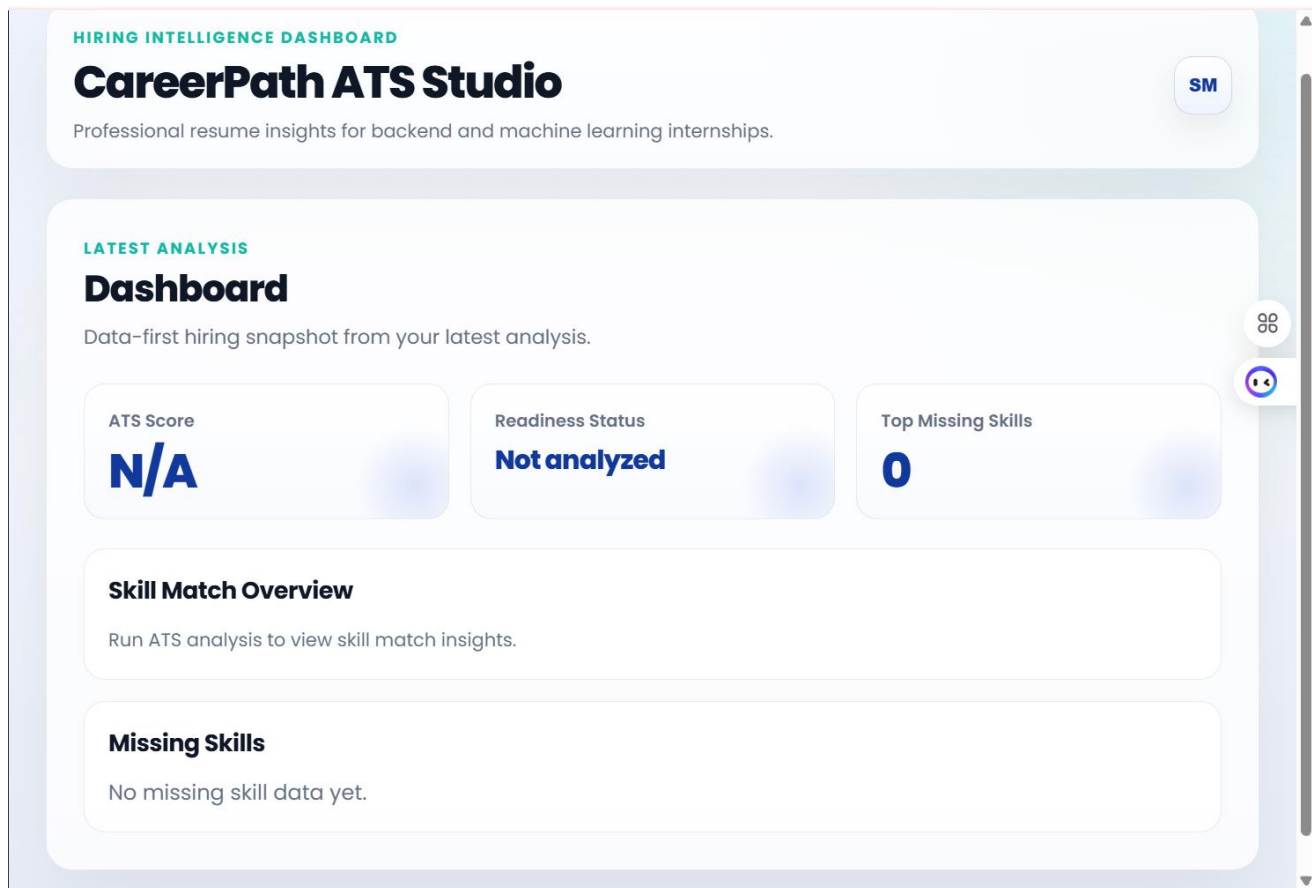


Figure 5.3: Dashboard UI

HIRING INTELLIGENCE DASHBOARD

CareerPath ATS Studio

SM

Professional resume insights for backend and machine learning internships.

Upload Resume

Upload a PDF or DOCX file and run ATS analysis. Interests/goal are optional (the system can infer them).

Choose file No file chosen

Interests (e.g., backend, ML, cloud)

Goal (e.g., internship, entry-level job)

Tip: Interests/goal help, but if you leave them empty we infer them from your resume and still recommend career paths.

Analyze Resume

Upload resume and start ATS analysis.



Figure 5.4: Upload Resume Page

Chapter 6

Implementation

6.1 Development Environment

The project was developed using Python and Flask for backend logic. The frontend was created using HTML, CSS, and vanilla JavaScript. Resume files are processed on the backend, and responses are returned as JSON to the frontend. The development environment requires Python, pip, and a browser.

6.2 Backend Implementation

The backend file app.py contains the core functionality of the system. It defines system prompts for ATS analysis, resume improvements, skill gap analysis, and learning roadmap generation. It also contains routes for uploading resumes, running analysis stages, downloading reports, retrieving previous analysis, and viewing template suggestions.

Backend Function / Route	Purpose
extract text from pdf()	Extracts text from PDF files.
extract text from docx()	Extracts text from DOCX files.
extract resume text()	Validates file type and extracts resume text.
call_grok()	Sends stage prompt and resume text to the AI model API.
/upload	Uploads resume and runs initial ATS analysis.
/analyze/<stage>	Runs improvements, skill gap, or roadmap stage.
/download-improved/<analysis_id>/<file_format>	Downloads improved resume as PDF or DOCX.
/download-roadmap-pdf/<analysis_id>	Downloads generated roadmap as PDF.
/get-template-suggestions/<analysis_id>	Returns template and formatting suggestions.

app.py > build_fallback_career_paths

```
1  import time
2  import uuid
3
4  from flask import Flask, jsonify, render_template, request, send_file
5  from dotenv import load_dotenv
6
7  from database.storage import get_or_load_state, resume_store, save_analysis_state
8  from prompts.stage_schemas import NEXT_STAGE
9  from prompts.system_prompts import SYSTEM_PROMPTS
10 from rag.retriever import retrieve_kb_context
11 from reports.pdf_report import build_analysis_report_pdf
12 from resume.parser import extract_resume_text
13 from utils.grok_client import call_grok
14 from utils.uuid_utils import UUID_RE
15
16
17 load_dotenv()
18
19 app = Flask(__name__)
20
21
22 @app.route("/", methods=["GET"])
23 def index():
24     return render_template("index.html", asset_version=str(int(time.time())))
25
26
27 @app.route("/upload", methods=["POST"])
28 def upload_resume():
29     if "resume" not in request.files:
30         return jsonify({"error": "No file uploaded."}), 400
31
32     uploaded_file = request.files["resume"]
33     if not uploaded_file or uploaded_file.filename == "":
34         return jsonify({"error": "Invalid file."}), 400
```

6.3 Frontend Implementation

The frontend uses index.html to define the interface structure, styles.css for visual design, and app.js for dynamic behavior. JavaScript handles resume upload, stage switching, API requests, rendering of analysis results, dashboard updates, and file download actions.

Frontend File	Implementation Details
index.html	Defines sidebar, dashboard, upload section, result section, and action buttons.
app.js	Uses fetch API to upload resumes, run analysis stages, and download files.
styles.css	Defines layout, cards, buttons, sidebar, responsive view, and color styling.

6.4 AI Prompt Design

The system uses separate strict prompts for each analysis stage. This improves output consistency because each prompt has a specific purpose. ATS prompt focuses on scoring, improvements prompt focuses on resume correction, skill gap prompt focuses on employability weaknesses, and roadmap prompt converts gaps into a practical learning plan.

Prompt Stage	Purpose
ATS	Generates ATS score, breakdown, strengths, weaknesses, and score reasoning.
Improvements	Provides resume content, formatting, keyword, and project improvement suggestions.
Skill Gap	Identifies missing skills and weaknesses for internship readiness.
Roadmap	Creates an 8 to 12 week learning plan with projects and outcomes.

6.5 Report Generation

The project includes report generation features. The improved resume can be downloaded as PDF or DOCX, while the learning roadmap can be downloaded as a formatted PDF. The report generation functions use reportlab for PDF and python-docx for DOCX output.

```
1  from datetime import datetime
2  from io import BytesIO
3
4  from reportlab.lib import colors
5  from reportlab.lib.pagesizes import letter
6  from reportlab.lib.styles import ParagraphStyle, getSampleStyleSheet
7  from reportlab.platypus import (
8      ListFlowable,
9      ListItem,
10     PageBreak,
11     Paragraph,
12     SimpleDocTemplate,
13     Spacer,
14 )
15
16
17  def build_analysis_report_pdf(analysis_id, state):
18      buffer = BytesIO()
19
20      def clean_text(value):
21          text = str(value or "")
22          text = (
23              text.replace("\u25a0", "-")
24              .replace("\u2013", "-")
25              .replace("\u2014", "-")
26              .replace("\u2019", "'")
27              .replace("\u201c", '"')
28              .replace("\u201d", '"')
29              .replace("\u00a0", " ")
30          )
31          return " ".join(text.split())
32
33      styles = getSampleStyleSheet()
```

Chapter 7

Testing and Results

7.1 Testing Strategy

Testing was performed to verify that the application accepts valid resume files, rejects invalid input, extracts text properly, communicates with the AI API, displays results, and downloads output files. Both functional testing and user interface testing are important for this project.

7.2 Test Cases

Test Case	Input	Expected Result	Status
Upload PDF Resume	Valid .pdf file	Resume text extracted and ATS result displayed	Pass
Upload DOCX Resume	Valid .docx file	Resume text extracted and ATS result displayed	Pass
Invalid File Upload	Unsupported file type	Error message displayed	Pass
Run Improvements	Valid analysis ID	Improvement suggestions displayed	Pass
Run Skill Gap	Valid analysis ID	Missing skills and gaps displayed	Pass
Generate Roadmap	Valid previous analysis	Roadmap phases displayed	Pass
Download PDF	Improved resume available	PDF file downloaded	Pass
Download DOCX	Improved resume available	DOCX file downloaded	Pass

7.3 Output Screens

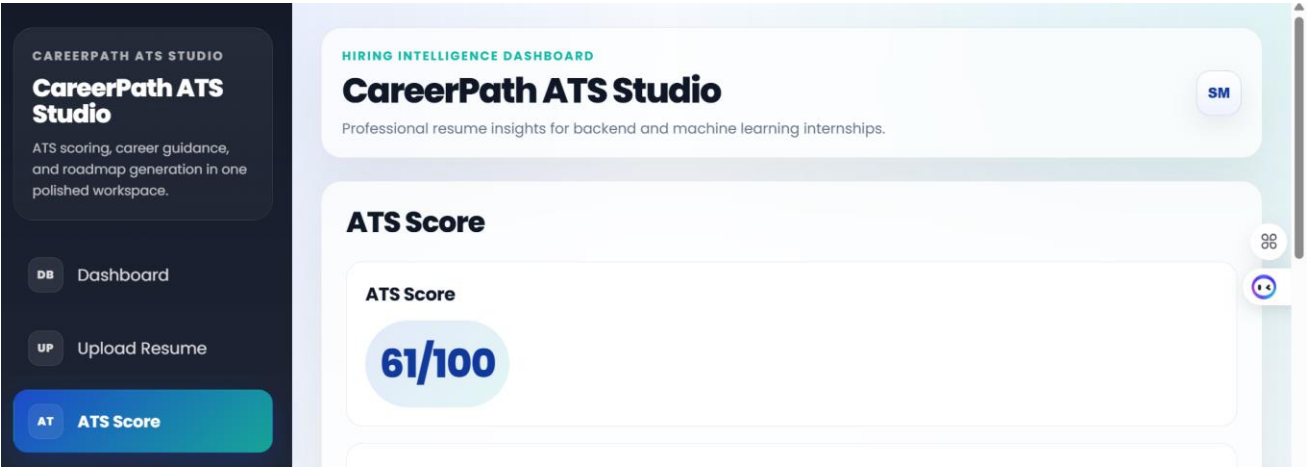


Figure 7.1: ATS Score Result

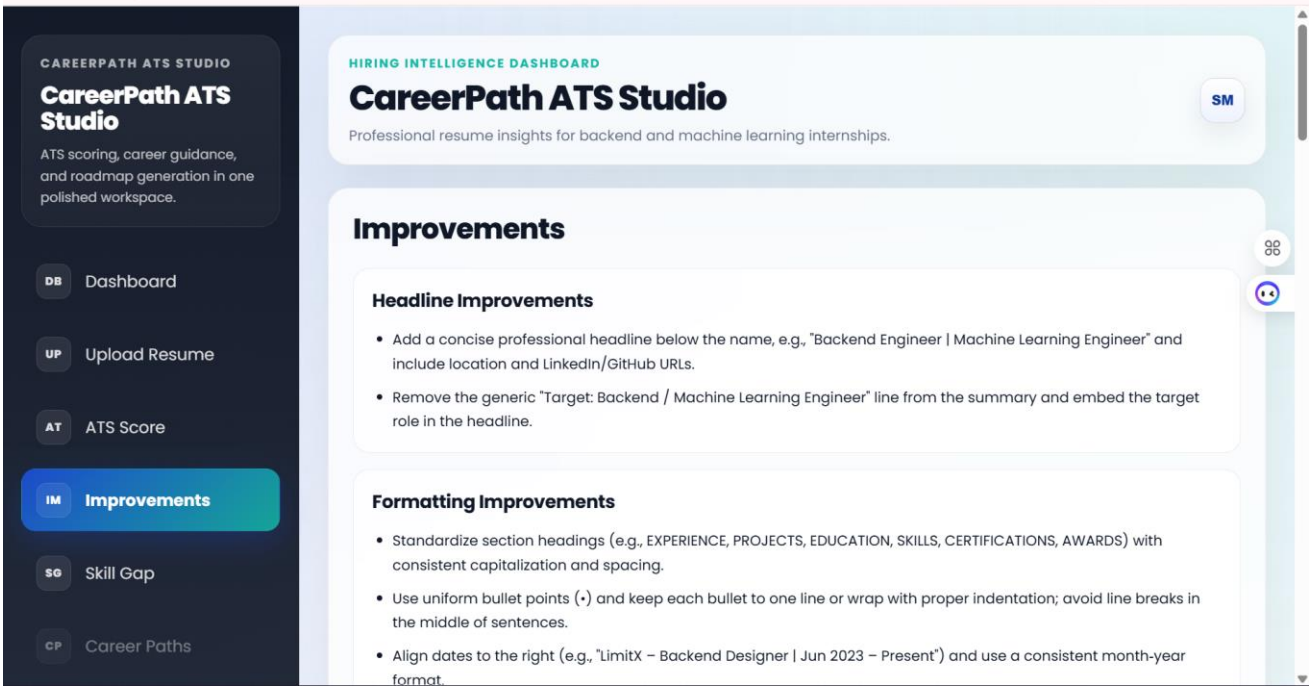


Figure 7.2: Improvements Result

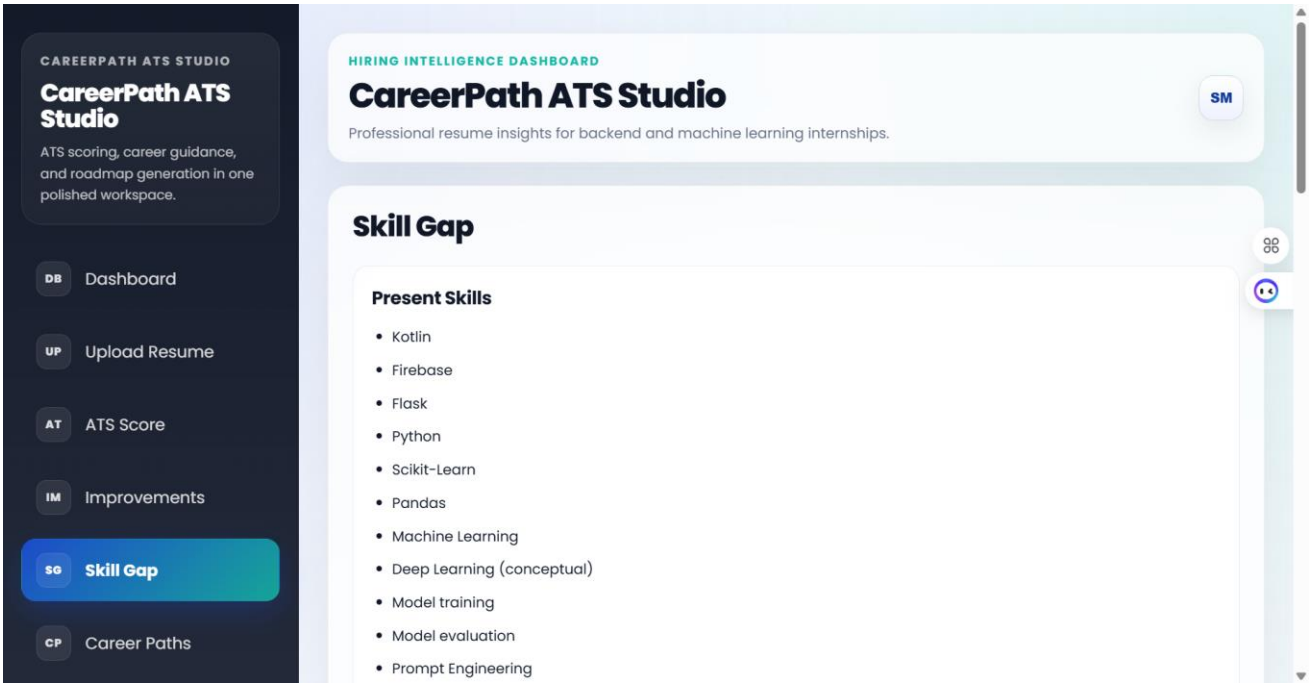


Figure 7.3: Skill Gap Result

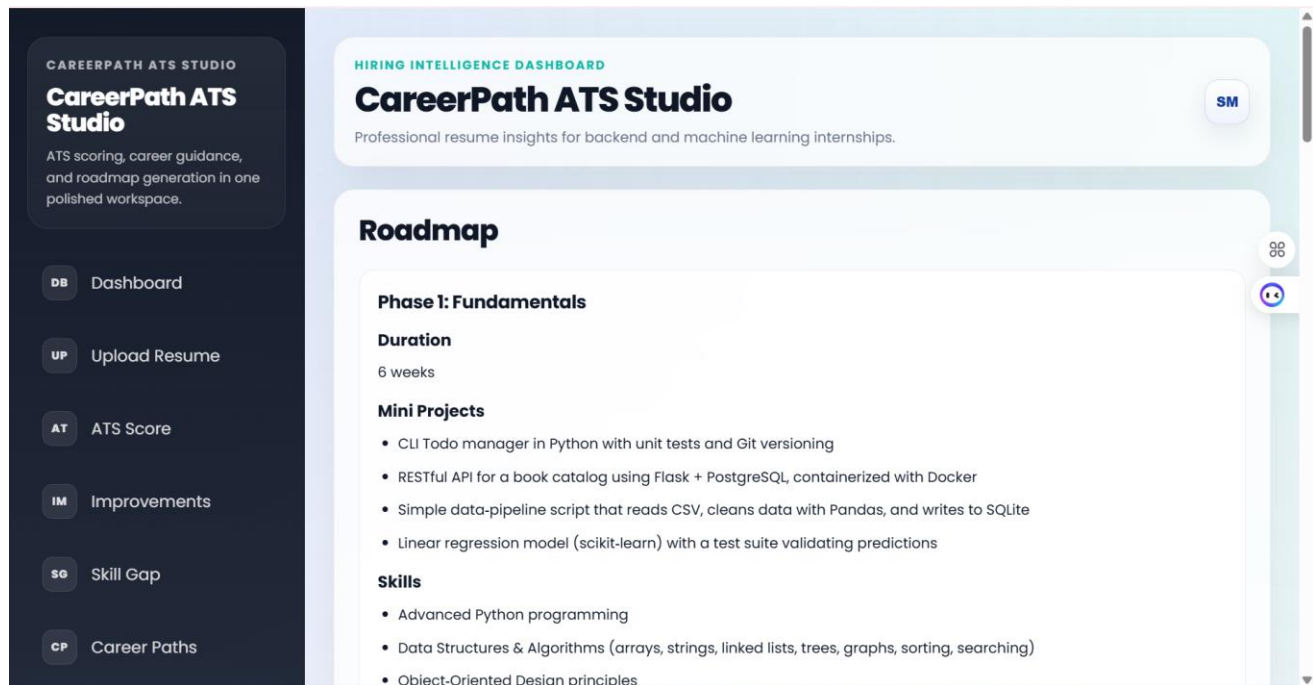


Figure 7.4: Roadmap Result

7.4 Result Discussion

The system successfully performs the core workflow of resume analysis. It accepts a resume, extracts content, returns structured AI results, updates the interface, and provides downloadable files. The modular stage-based design makes the system easy to understand and extend. Since results depend on AI API availability and prompt quality, stable internet connection and valid API configuration are necessary.